

Assignment 3: Bridge Pattern Report

Name: Sultanov Abilkaiyr

Group: SE-2529

Topic: B (Notifications)

1. Two Independent Dimensions and Bridge Rationale

In the chosen topic (Notifications), the application contains two independently varying dimensions:

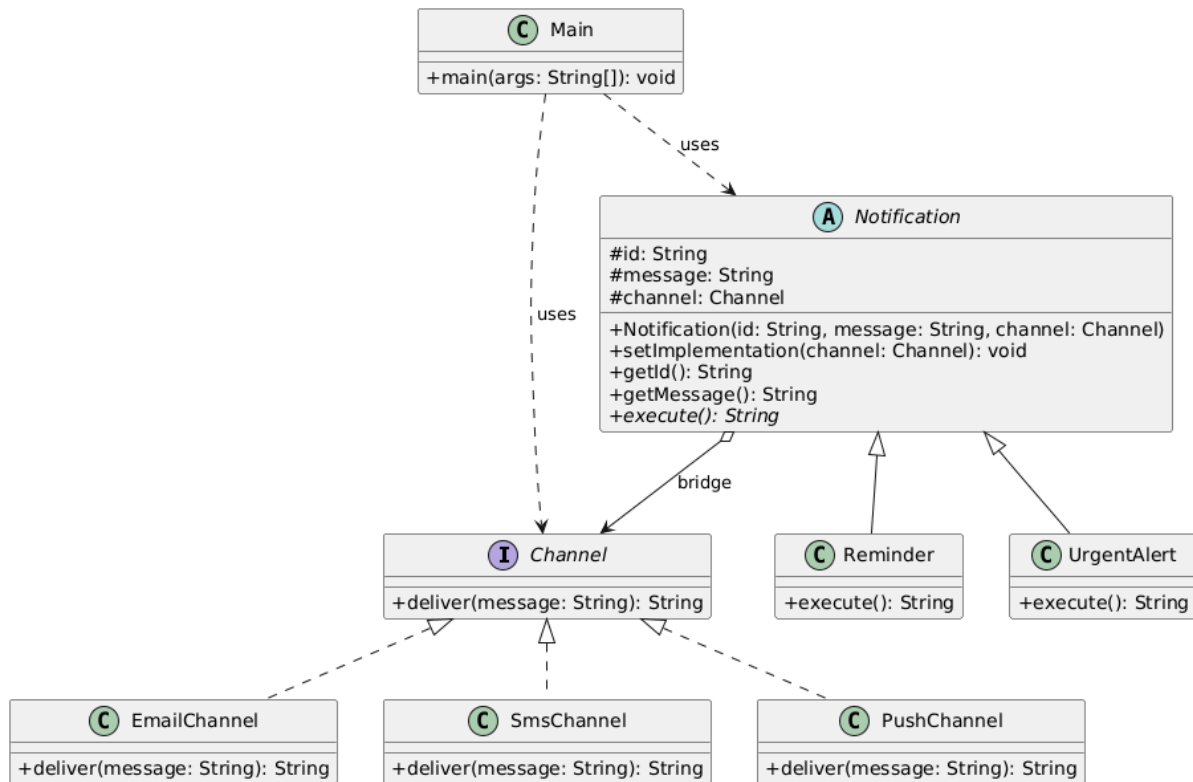
1. The Abstraction (Notification Types): The logical types of notifications, such as `Reminder` and `UrgentAlert`.
2. The Implementation (Delivery Channels): The actual delivery mechanisms, such as `EmailChannel`, `SmsChannel`, and later `PushChannel`.

The Bridge pattern fits perfectly because it decouples an abstraction from its implementation so that the two can vary independently. Instead of creating a Cartesian product of subclasses (e.g., `EmailReminder`, `SmsUrgentAlert`), Bridge uses composition. The `Notification` class holds a reference to a `Channel` interface, allowing us to mix and match notifications and delivery methods dynamically at runtime.

2. Bridge vs. Adapter

While both structural patterns deal with connecting components, their intent differs significantly. The Bridge pattern is applied up-front during the design phase to separate orthogonal dimensions of a system (abstraction and implementation) to let them evolve independently. The Adapter pattern, on the other hand, is generally applied after a system is designed to make two existing, incompatible interfaces work together.

3. UML Class Diagram



4. Clean Code Practices

Below are five annotated code excerpts demonstrating Clean Code principles used in the implementation:

1. Separation of Responsibilities: *Excerpt:* `public interface Channel { String deliver(String message); }` *Benefit:* The Channel interface and its implementors are strictly responsible for delivery formatting. They do not know about notification types, ensuring loose coupling.
2. Meaningful Names: *Excerpt:* `public class UrgentAlert extends Notification` *Benefit:* The class name explicitly states its domain purpose, making the code self-documenting and easier for other developers to understand without reading the underlying logic.
3. Small Focused Methods: *Excerpt:* `@Override public String execute() { return channel.deliver("[URGENT] " + message); }` *Benefit:* Methods are kept extremely short and perform exactly one task. This makes debugging straightforward and reduces complexity.
4. Avoidance of Duplication: *Excerpt:* `super(id, message, channel);` (inside **Reminder** and **UrgentAlert** constructors). *Benefit:* By centralizing the assignment of `id`, `message`, and the `channel` reference in the abstract base class **Notification**, we eliminate redundant boilerplate code in the concrete subclasses.
5. Encapsulation of State: *Excerpt:* `protected String id; protected String message;` *Benefit:* The state of the abstraction is hidden from external classes. It is only accessible via controlled getters (`getId()`, `getMessage()`), protecting the object's integrity.

5. Independent Extension (Adding I3) and Trade-offs

Extension Process: To add the new PushChannel (I3), I simply created a new class implementing the Channel interface. The existing Notification hierarchy, the Channel interface itself, EmailChannel, and SmsChannel remained completely unchanged. The only other file modified was the Main client to test the new combinations (T6, T7).

Trade-off of Bridge: One trade-off of the Bridge pattern is that it increases the initial complexity of the code. It requires designing an abstract interface and setting up composition, which can be over-engineering for simple applications where dimensions are not expected to change or grow.

References

- Astana IT University, Course Syllabus: ShP-2216 Software Design Patterns (2026-2027).
- Lecture 4, Bridge Pattern.
- Bridge reading materials provided in Moodle.